```
function [retmap,retvisited,retsteps] = greedy(mapfile,startlocation,targetlocation)

% Convert the map file into a MATLAB matrix
map = map_convert(mapfile);
retmap = map;

% Get dimensions of the map
[maprows,mapcols] = size(map);

% Keep track of visited cells
visited = false(maprows,mapcols);

% Keep track of the parent of every visited cell
parent = zeros(maprows,mapcols,2);

% Store cells that still need to be explored
openSet = startlocation;

% Store the order in which cells are visited
retvisited = [];

% Directions:
%       up
%       down
%       left
%       right
directions = [
    -1  0
     1  0
     0 -1
     0  1
];

% Mark the starting location as visited
visited(startlocation(1),startlocation(2)) = true;

% Greedy Best-First Search
while ~isempty(openSet)

    % Find the cell with the smallest heuristic
    bestIndex = 1;
    bestHeuristic = inf;

    for i = 1:size(openSet,1)

        row = openSet(i,1);
        col = openSet(i,2);

        % Manhattan distance to target
        heuristic = ...
            abs(row - targetlocation(1)) + ...
            abs(col - targetlocation(2));

        if heuristic < bestHeuristic

            bestHeuristic = heuristic;
            bestIndex = i;

        end
    end

```

```
    % Select the cell with the smallest heuristic
    current = openSet(bestIndex,:);

    % Remove it from the open set
    openSet(bestIndex,:) = [];

    % Record the cell as visited
    retvisited = [retvisited; current];

    % Have we reached the target?
    if current(1) == targetlocation(1) && ...
       current(2) == targetlocation(2)

        break;
    end

    % Check neighbouring cells
    for k = 1:4

        neighbour = current + directions(k,:);

        row = neighbour(1);
        col = neighbour(2);

        % Check that neighbour is inside the map
        if row >= 1 && row <= maprows && ...
           col >= 1 && col <= mapcols

            % Check that it is not a wall
            if map(row,col) == 0

                % Check that it has not already been visited
                if ~visited(row,col)

                    % Mark as visited
                    visited(row,col) = true;

                    % Remember where we came from
                    parent(row,col,1) = current(1);
                    parent(row,col,2) = current(2);

                    % Add neighbour to open set
                    openSet = [openSet; neighbour];

                end
            end
        end
    end
end

% Reconstruct path
retsteps = [];

% Check whether target was reached
if visited(targetlocation(1),targetlocation(2))

    current = targetlocation;

    while ~(current(1) == startlocation(1) && ...
            current(2) == startlocation(2))

```

```
123         % Add current cell to path
124         retsteps = [current; retsteps];
125 
126         % Move to parent
127         parentrow = parent(current(1),current(2),1);
128         parentcol = parent(current(1),current(2),2);
129 
130         current = [parentrow parentcol];
131 
132     end
133 
134     % Add starting location
135     retsteps = [startlocation; retsteps];
136 
137 end
138 
139 plotmap(retmap, retsteps);
140 
141 end
142 
143 
144 function placestep(position,i)
145 
146 % This function will plot a yellow rectangle
147 % and print a number in the rectangle.
148 
149 position = [16-position(1) position(2)];
150 
151 position = [position(2)+0.1 position(1)+0.1];
152 
153 rectangle('Position',[position,0.8,0.8],'FaceColor','y');
154 
155 c = sprintf('%d',i);
156 
157 text(position(1)+0.2,position(2)+0.2,c,'FontSize',10);
158 
159 end
```